План курса

Часть 1. Введение

Тем: 4

Тема 1. Ввод-вывод

Содержание

1. Потоки ввода-вывода
   1. Четыре вида потоков
   2. Операции с потоками
   3. Исключения ввода-вывода
2. Файловый ввод-вывод и преобразование потоков
   1. Классы файловых потоков
   2. Кодировки и преобразование потоков
3. Фильтрующие потоки
   1. Фильтрация
   2. Буферизация
4. Дополнительные возможности потоков
   1. Эмуляция чтения и записи
   2. Конкатенация потоков ввода
   3. Подавление ошибок
5. Расширенный ввод-вывод
   1. Платформонезависимый ввод-вывод
   2. Файлы с произвольным доступом
   3. Классы System и Console
6. Дескрипторы файлов
   1. Файлы, каталоги, пути и разделители
   2. Операции с отдельными файлами
   3. Листинг каталогов
7. Ввод-вывод в Java 7
   1. Файловые системы и пути
   2. Операции с отдельными файлами
   3. Листинг каталогов
8. Ввод-вывод и исключения
   1. Идиома работы с ресурсами
   2. Работа с несколькими потоками

Слайды

HTML

Примеры

zip

Домашнее задание 1. Обход файлов

1. Разработайте класс `Walk`, осуществляющий подсчет хэш-сумм файлов.
   1. Формат запуска:
      `java Walk <входной файл> <выходной файл>`
   2. Входной файл содержит список файлов, которые требуется обойти.
   3. Выходной файл должен содержать по одной строке для каждого файла. Формат строки:
      `<шестнадцатеричная хэш-сумма> <путь к файлу>`
   4. Для подсчета хэш-суммы используйте алгоритм FNV.
   5. Если при чтении файла возникают ошибки, укажите в качестве его хэш-суммы все нули.
   6. Кодировка входного и выходного файлов — UTF-8.
   7. Размеры файлов могут превышать размер оперативной памяти.
   8. Пример

Входной файл
samples/1
samples/12
samples/123
samples/1234
samples/1
samples/binary
samples/no-such-file

Выходной файл
050c5d2e samples/1
2076af58 samples/12
72d607bb samples/123
81ee2b55 samples/1234
050c5d2e samples/1
8e8881c5 samples/binary
00000000 samples/no-such-file

2. Сложный вариант:
   1. Разработайте класс `RecursiveWalk`, осуществляющий подсчет хэш-сумм файлов в директориях.
   2. Входной файл содержит список файлов и директорий, которые требуется обойти. Обход директорий осуществляется рекурсивно.
   3. Пример:

Входной файл
samples/binary
samples
samples/no-such-file

Выходной файл
8e8881c5 samples/binary
050c5d2e samples/1
2076af58 samples/12
72d607bb samples/123
81ee2b55 samples/1234
8e8881c5 samples/binary
00000000 samples/no-such-file

3. При выполнении задания следует обратить внимание на:
   - Дизайн и обработку исключений, диагностику ошибок.
   - Программа должна корректно завершаться даже в случае ошибки.
   - Корректная работа с вводом-выводом.
   - Отсутствие утечки ресурсов.
   - Возможность повторного использования кода.
4. Требования к оформлению задания.
   - Проверяется исходный код задания.
   - Весь код должен находиться в пакете `info.kgeorgiy.ja.фамилия.walk`.

Репозиторий с тестами

Библиография

1. Guide: The Java I/O API
2. Guide: Common I/O Tasks in Modern Java
3. Java I/O, NIO, and NIO.2
4. JEP 400: UTF-8 by Default

Тема 2. Collections Framework

Содержание

1. Контейнеры
   1. Коллекции
      1. Интерфейс Collection
      2. Итераторы
      3. Класс AbstractCollection
   2. Множества
      1. Интерфейс Set
      2. Методы equals и hashCode
      3. Реализации множеств и базовые классы
   3. Списки
      1. Интерфейс List
      2. Итераторы по спискам
      3. Реализации списков и базовые классы
   4. Очереди и деки
      1. Интерфейс Queue
      2. Интерфейс Dequeue
      3. Реализации очередей и деков, базовые классы
2. Отображения
   1. Интерфейс Map
   2. Пары и проекции отображений
   3. Реализации отображений и базовые классы
3. Упорядоченные коллекции
   1. Сравнение элементов и согласованность с equals
   2. Упорядоченные множества
   3. Упорядоченные отображения
   4. Очередь с приоритетами
4. Алгоритмы
   1. Класс Collections
   2. Класс Arrays
5. Устаревшие коллекции

Слайды

HTML

Примеры

zip

Домашнее задание 2. Множество на массиве

1. Разработайте класс `ArraySet`, реализующий неизменяемое упорядоченное множество.
   - Класс `ArraySet` должен реализовывать интерфейс SortedSet (простой вариант) или NavigableSet (сложный вариант).
   - Все операции над множествами должны производиться с наилучшей асимптотической эффективностью.
2. При выполнении задания следует обратить внимание на:
   - Применение стандартных коллекций.
   - Избавление от повторяющегося кода.
   - Отсутствие unchecked warnings при компиляции.
   - Отсутствие излишних подавленных unchecked warnings.

Библиография

1. Guide: The Collections Framework
2. Outline of the Collections Framework
3. Java Collections Cheat Sheet

Тема 3. Лямбда-выражения и потоки

Содержание

1. Расширения интерфейсов
2. Лямбда-выражения
   1. Функциональные интерфейсы
   2. Лямбда-выражения
   3. Ссылки на методы
3. Поддержка лямбда-выражений в основных классах
   1. Типы-обертки
   2. Класс Objects
   3. Стандартные функциональные интерфейсы
4. Поддержка лямбда-выражений в Collections Framework
   1. Необязательные значения
   2. Компараторы
   3. Новые методы коллекций и отображений
5. Потоки значений
   1. Применение
      1. Типы потоков
      2. Создание и преобразование потоков
      3. Комбинаторы
   2. Стандартные коллекторы
   3. Реализация потоков
      1. Разбиваемые итераторы
      2. Обобщенные свертки
      3. Параллельное исполнение

Слайды

HTML

Примеры

zip

Домашнее задание 3. Студенты

1. Разработайте класс `StudentDB`, осуществляющий поиск по базе данных студентов.
   - Класс `StudentDB` должен реализовывать интерфейс `StudentQuery` (простой вариант) или `GroupQuery` (сложный вариант).
   - Каждый метод должен состоять из ровно одного оператора. При этом длинные операторы надо разбивать на несколько строк.
2. При выполнении задания следует обратить внимание на:
   - применение лямбда-выражений и потоков;
   - избавление от повторяющегося кода.

Домашнее задание 4. Реализация потоков

1. Разработайте класс `Streams`, реализующий сплитераторы для деревьев и дополнительные коллекторы.
   - *Простой вариант* (интерфейс `EasyStreams`) — реализуйте:
     - сплитераторы для двоичных деревьев, двоичных деревьев с известным размером, k-ичных деревьев;
     - коллекторы первого, последнего, среднего элементов;
     - коллекторы общего префикса и суффикса строк;
     - преобразователь строки в её непустые префиксы;
     - преобразователь, оставляющий каждый *n*-й элемент;
     - преобразователь, оставляющий префикс потока состоящий из уникальных элементов.
   - *Сложный вариант* (интерфейс `HardStreams`) — дополнительно реализуйте:
     - сплитераторы всех видов деревьев над списками элементов;
     - коллектор *k*-ого элемента;
     - коллекторы первых и последних *n* элементов.
     - преобразователь строки в её непустые суффиксы;
     - преобразователь, оставляющий `i`-й из каждого *n*-элементного окна;
     - преобразователь, оставляющий префикс потока, для которых функция выдаёт уникальные элементы.
2. При выполнении задания следует обратить внимание на:
   - характеристики создаваемых сплитераторов;
   - избавление от повторяющегося кода.

Библиография

1. Guide: Lambda Expressions
2. Guide: The Stream API
3. Пакет java.util.stream
4. Пакет java.util.function
5. JEP 485: Stream Gatherers

Тема 4. Reflection

Содержание

1. Введение в Reflection
   1. Идентификаторы типов (type tokens)
   2. Информация о типе
2. Структура класса
   1. Поля
      1. Получение информации
      2. Чтение и запись значений
   2. Методы и конструкторы
      1. Получение информации
      2. Вызов методов
      3. Создание объектов
   3. Вложенные классы
3. Массивы
4. Загрузка классов
   1. Загрузчики классов
   2. Иерархия загрузчиков
   3. Доступ к ресурсам
5. Параметры типов
   1. Интерфейс Type и его потомки
   2. Generic-массивы
6. Посредники
   1. Создание посредников
   2. Обработчики вызовов
   3. Применение посредников

Слайды

HTML

Примеры

zip

Домашнее задание 5. Implementor

1. Реализуйте класс `Implementor`, генерирующий реализации классов и интерфейсов.
   - Аргумент командной строки: полное имя класса/интерфейса, для которого требуется сгенерировать реализацию.
   - В результате работы должен быть сгенерирован java-код класса с суффиксом `Impl`, расширяющий (реализующий) указанный класс (интерфейс).
   - Сгенерированный класс должен компилироваться без ошибок.
   - Сгенерированный класс не должен быть абстрактным.
   - Методы сгенерированного класса должны игнорировать свои аргументы и возвращать значения по умолчанию.
2. В задании выделяются три варианта:
   - *Простой* — `Implementor` должен уметь реализовывать только интерфейсы (но не классы). Поддержка generics не требуется.
   - *Сложный* — `Implementor` должен уметь реализовывать и классы, и интерфейсы. Поддержка generics не требуется.
   - *Бонусный* — `Implementor` должен уметь реализовывать generic-классы и интерфейсы. Сгенерированный код должен иметь корректные параметры типов и не порождать unchecked warnings.

Библиография

1. Guide: The Reflection API (guide)
2. Guide: Introduction to Java Reflection
3. Guide: Introduction to Method Handles
4. *Forman I.R., Forman N.* Java Reflection in Action

Часть 2. Инструменты

Тем: 3

Тема 5. Jar

Содержание

1. Jar-файлы
   1. Инструмент командной строки
   2. API для работы с Zip, Jar и Gzip
2. Манифесты
   1. Формат
   2. Основные атрибуты

Слайды

HTML

Домашнее задание 6. Jar Implementor

Это домашнее задание **связано** с предыдущим и будет приниматься только с ним. Предыдущее домашнее задание отдельно сдать будет нельзя.

1. Создайте `.jar`-файл, содержащий скомпилированный `Implementor` и сопутствующие классы.
   - Созданный `.jar`-файл должен запускаться командой `java -jar`.
   - Запускаемый `.jar`-файл должен принимать те же аргументы командной строки, что и класс `Implementor`.
2. Модифицируйте `Implementor` так, чтобы при запуске с аргументами `-jar имя-класса файл.jar` он генерировал `.jar`-файл с реализацией соответствующего класса (интерфейса). Для компиляции можно использовать код из тестов.
3. Вы можете создавать файлы и директории в текущем каталоге, но не за его пределами.
4. Для проверки, кроме исходного кода, также должны быть представлены:
   - скрипт для создания запускаемого `.jar`-файла, в том числе, исходный код манифеста;
   - запускаемый `.jar`-файл.
5. **Сложный вариант**. Решение должно быть модуляризовано.

Библиография

1. JAR File Specification
2. Guide: Jar - the Archive Tool
3. Guide: Jarsigner - Signing Your JARs
4. Tutorial: Packaging Programs in JAR Files

Тема 6. Javadoc

Содержание

1. Что такое Javadoc
2. Структура Javadoc
   1. Javadoc-комментарии
   2. Типы тегов
3. Блочные теги
   1. Общие теги
   2. Теги методов
4. Строчные теги
   1. Ссылки
   2. Код и значения
5. Применение Javadoc
   1. Объекты документирования
   2. Наследование Javadoc
6. Компиляция Javadoc

Слайды

HTML

Примеры

zip

Домашнее задание 7. Javadoc

Это домашнее задание **связано** с двумя предыдущими и будет приниматься только с ними. Предыдущие домашние задания отдельно сдать будет нельзя.

1. Документируйте класс `Implementor` и сопутствующие классы с применением Javadoc.
   - Должны быть документированы все классы и все члены классов, в том числе `private`.
   - Документация должна генерироваться без предупреждений.
   - Сгенерированная документация должна содержать корректные ссылки на классы стандартной библиотеки и модулей `info.kgeorgiy.java.advanced.*`.
2. Для проверки, кроме исходного кода, также должны быть представлены:
   - скрипт для генерации документации (он может рассчитывать, что рядом с вашим репозиторием склонирован репозиторий курса);
   - сгенерированная документация.

В **последующих** домашних заданиях все `public` и `protected` сущности должны быть документированы.

Библиография

1. Javadoc Guide
2. How to Write Doc Comments for the Javadoc Tool
3. Guide: Javadoc - the Documentation Generator
4. Javadoc FAQ

Тема 7. Модули

Содержание

1. Что такое модули
   1. Определение
   2. Объявление
   3. Module path
2. Управление зависимостями
   1. Подключение зависимостей
   2. Экспорт пакетов
   3. Открытие пакетов
3. Сервисы
   1. Определение сервиса
   2. Реализация сервиса и ее объявление
   3. Объявление использования сервиса
   4. Загрузка сервисов
4. Модуляризованный JDK

Слайды

HTML

Примеры

zip

Библиография

1. JLS: Module Declaration
2. Javadoc: Service Loader
3. Guide: Modules
4. Creating Runtime and Application Images with JLink

Часть 3. Многопоточное программирование

Тем: 4

Тема 8. Введение в многопоточное программирование

Содержание

1. Параллельное исполнение
   1. Развитие параллельных систем
   2. Потоки и процессы
   3. Типы параллелизма
      1. Итеративный
      2. Рекурсивный
      3. Обмен сообщениями
   4. Неделимые операции
      1. Безусловные
      2. Условные
      3. Свойства планировщика
2. Потоки
   1. Создание потоков
   2. Состояния и свойства потоков
   3. Завершение потоков
   4. Механизм прерывания

Слайды

HTML

Примеры

zip

Домашнее задание 8. Итеративный параллелизм

1. Реализуйте класс `IterativeParallelism`, который будет обрабатывать списки в несколько потоков.
2. В простом варианте должны быть реализованы следующие методы:
   - `argMax(threads, list, comparator)` — индекс первого максимума;
   - `argMin(threads, list, comparator)` — индекс первого минимума;
   - `indexOf(threads, list, predicate)` — индекс первого элемента, удовлетворяющего предикат;
   - `lastIndexOf(threads, list, predicate)` — индекс последнего элемента, удовлетворяющего предикат;
   - `sumIndices(threads, list, predicate)` — сумма индексов элементов, удовлетворяющих предикат;
3. В сложном варианте должны быть дополнительно реализованы следующие методы:
   - `indices(threads, list, predicate)` — индексы элементов, удовлетворяющих предикат;
   - `filter(threads, list, predicate)` — вернуть список, содержащий элементы удовлетворяющие предикату;
   - `map(threads, list, function)` — вернуть список, содержащий результаты применения функции;
4. Во все функции передается параметр `threads` — сколько потоков надо использовать при вычислении. Вы можете рассчитывать, что число потоков относительно мало.
5. Не следует рассчитывать на то, что переданные компараторы, предикаты и функции работают быстро.
6. Можно сделать O(`threads`), но не O(`list.size()`) действий без распараллеливания.
7. При выполнении задания **нельзя** использовать *Concurrency Utilities* и *Parallel Streams*.

Библиография

1. Java Language Specification. Theads and locks
2. Concurrency (Java Tutorial)
3. *Lea D.* Concurrent Programming in Java
4. *Goetz B.* Java Concurrency in Practice
5. *Garg V.* Concurrent and Distributed Computing in Java

Тема 9. Взаимодействие потоков

Содержание

1. Синхронизация и блокировки
   1. Синхронизация
   2. Неявные блокировки
   3. Активное ожидания
2. Мониторы и условия
   1. Мониторы
   2. Условия и работа с ними
   3. Пассивное ожидание
   4. Внезапные пробуждения
3. Модель памяти Java
   1. Атомарность операций
   2. Видимость изменений и барьеры
   3. Упорядоченность изменений
   4. Volatile-переменные
4. Примеры многопоточных программ
   1. Двусторонний барьер
   2. Гарантированный тупик
5. Уровни безопасности
   1. Неизменяемые объекты
   2. Потокобезопасные объекты
   3. Потоконебезопасные объекты
   4. Thread-local objects

Слайды

HTML

Примеры

zip

Домашнее задание 9. Параллельный запуск

1. Напишите класс `ParallelMapperImpl`, реализующий интерфейс `ParallelMapper`.
   ```
   public interface ParallelMapper extends AutoCloseable {
       <T, R> List<R> map(
           Function<? super T, ? extends R> f,
           List<? extends T> args
       ) throws InterruptedException;

       @Override
       void close();
   }
   ```
   - Метод `map` должен параллельно вычислять функцию `f` на каждом из указанных аргументов (`args`).
   - Конструктор `ParallelMapperImpl(int threads)` должен создавать `threads` рабочих потоков, которые используются для распараллеливания.
   - Метод `close` должен останавливать все рабочие потоки.
   - К одному `ParallelMapperImpl` могут одновременно обращаться несколько клиентов.
   - При недостатке потоков для распараллеливания, задания на исполнение должны накапливаться в очереди и обрабатываться в порядке поступления.
   - В реализации не должно быть активных ожиданий.
   - Код должен находиться в пакете `iterative`.
   - Обратите внимание на обработку исключений, кидаемых функцией `f`.
     1. Исключения не должны приводить к сокращению числа рабочих потоков.
     2. **Сложный вариант.** Исключения должны выкидываться из метода `map`.
2. Доработайте класс `IterativeParallelism` так, чтобы он мог использовать `ParallelMapper`.
   - Добавьте конструктор `IterativeParallelism(ParallelMapper)`.
   - Методы класса должны делить работу на `threads` фрагментов и исполнять их при помощи `ParallelMapper`.
   - При наличии `ParallelMapper` сам `IterativeParallelism` новые потоки создавать не должен.
   - Должна быть возможность одновременного запуска и работы нескольких клиентов, использующих один `ParallelMapper`.
3. При выполнении задания всё ещё **нельзя** использовать *Concurrency Utilities* и *Parallel Streams*.

Библиография

1. Java Language Specification. Theads and locks
2. Concurrency (Java Tutorial)
3. *Lea D.* Concurrent Programming in Java
4. *Goetz B.* Java Concurrency in Practice
5. *Garg V.* Concurrent and Distributed Computing in Java

Тема 10. Задачи и средства многопоточного программирования

Содержание

1. Классические задачи многопоточного программирования
   1. Доступ к разделяемому ресурсу
   2. Производители и потребители
   3. Читатели и писатели
   4. Обедающие философы
   5. Задания и работники
2. Атомарные операции
   1. Виды атомарных операций
   2. CompareAndSet
   3. Применение атомарных операций
   4. Помеченные ссылки
3. Блокировки и условия
   1. Блокировки
   2. Условия
   3. Блокировки чтения-записи
4. Примитивы синхронизации
   1. Критические секции
   2. Семафоры
   3. Барьеры
   4. Защелки
   5. Рандеву
   6. Мониторы
5. Решение классических задач многопоточного программирования
   1. Читатели и писатели
   2. Обедающие философы
   3. Задания и работники

Слайды

HTML

Библиография

1. Java Concurrency Utilities
2. High Level Concurrency Objects (Java Tutorial)
3. JSR 166: Concurrency Utilities
4. Concurrent Programming with J2SE 5.0
5. *Эндрюс Г.* Основы многопоточного, параллельного и распределенного программирования
6. *Goetz B.* Java concurrency in practice

Тема 11. Concurrency Utilities

Содержание

1. Многопоточные коллекции
   1. Блокирующиеся очереди
   2. Многопоточные отображения
   3. Самокопирующиеся коллекции
2. Управление заданиями
   1. Исполнители
   2. Функции и результаты
   3. Отложенное исполнение
3. Прочие возможности
   1. Интервалы времени
   2. Очереди с задержкой

Слайды

HTML

Домашнее задание 10. Web Crawler

1. Напишите потокобезопасный класс `WebCrawler`, который будет рекурсивно обходить сайты.
   1. Класс `WebCrawler` должен иметь конструктор
      ```
      public WebCrawler(Downloader downloader, int downloaders, int extractors, int perHost)
      ```
      - `downloader` позволяет скачивать страницы и извлекать из них ссылки;
      - `downloaders` — максимальное число одновременно загружаемых страниц;
      - `extractors` — максимальное число страниц, из которых одновременно извлекаются ссылки;
      - `perHost` — максимальное число страниц, одновременно загружаемых с одного хоста. Для определения хоста следует использовать метод `getHost` класса `URLUtils` из тестов.
   2. Класс `WebCrawler` должен реализовывать интерфейс `Crawler`
      ```
      public interface Crawler extends AutoCloseable {
          Result download(String url, int depth);

          void close();
      }
      ```
      - Метод `download` должен рекурсивно обходить страницы, начиная с указанного URL, на указанную глубину и возвращать список загруженных страниц и файлов. Например, если глубина равна 1, то должна быть загружена только указанная страница. Если глубина равна 2, то указанная страница и те страницы и файлы, на которые она ссылается, и так далее.
      - Метод `download` может вызываться параллельно в нескольких потоках.
      - Загрузка и обработка страниц (извлечение ссылок) должна выполняться максимально параллельно, с учетом ограничений на число одновременно загружаемых страниц (в том числе с одного хоста) и страниц, с которых загружаются ссылки.
      - Для распараллеливания разрешается создать `downloaders + extractors` вспомогательных потоков.
      - Повторно загружать и/или извлекать ссылки из одной и той же страницы в рамках одного обхода (`download`) запрещается.
      - Метод `close` должен завершать все вспомогательные потоки.
   3. Для загрузки страниц должен применяться `Downloader`, передаваемый первым аргументом конструктора.
      ```
      public interface Downloader {
          public Document download(final String url) throws IOException;
      }
      ```
      - Метод `download` загружает документ по его адресу (URL).
      - Документ позволяет получить ссылки по загруженной странице:
        ```
        public interface Document {
            List<String> extractLinks() throws IOException;
        }
        ```
        Ссылки, возвращаемые документом, являются абсолютными и имеют схему `http` или `https`.
   4. Должен быть реализован метод `main`, позволяющий запустить обход из командной строки
      - Командная строка
        ```
        WebCrawler url [depth [downloaders [extractors [perHost]]]]
        ```
      - Для загрузки страниц требуется использовать реализацию `CachingDownloader` из тестов.
2. Версии задания
   1. *Простая* — не требуется учитывать ограничения на число одновременных закачек с одного хоста (`perHost >= downloaders`).
   2. *Полная* — требуется учитывать все ограничения.
   3. *Бонусная* — сделать параллельный обход в ширину без синхронизации между слоями.
3. Задание подразумевает активное использование Concurrency Utilities, в частности, в решении не должно быть «велосипедов», аналогичных/легко сводящихся к классам из Concurrency Utilities.

Библиография

1. JSR 166: Concurrency Utilities
2. Concurrent Programming with J2SE 5.0
3. Virtual Threads
4. Getting to know synchronizers
5. *Эндрюс Г.* Основы многопоточного, параллельного и распределенного программирования
6. *Goetz B.* Java concurrency in practice

Часть 4. Сетевые приложения

Тем: 3

Тема 12. Работа с сетью

Содержание

1. Введение
   1. Стек сетевых протоколов
   2. Протоколы IPv4 и IPv6
   3. Протокол TCP
   4. Протокол UDP
   5. DNS
2. Адреса
   1. Адреса хостов
   2. Работа с DNS
   3. Адреса портов
3. Работа с TCP
   1. Клиентский сокет
      1. Создание
      2. Управление ресурсами
      3. Закрытие части соединения
   2. Серверный сокет
      1. Создание
      2. Прием соединений
4. Работа с UDP
   1. Датаграммы
      1. Исходящие
      2. Входящие
   2. Сокеты
      1. Создание
      2. Отправка и прием данных
5. URI и URL
   1. URI
      1. Понятие об URI
      2. Структура URI
      3. Сравнение URI
   2. URL
      1. Понятие об URL
      2. Структура URL
      3. Сравнение URL
6. Соединения
   1. Создание соединения
   2. Жизненный цикл соединения
   3. Быстрое установление соединения
   4. Поддержка протоколов
7. Работа с HTTP
   1. Клиент
   2. Сервер

Слайды

HTML

Примеры

zip

Домашнее задание 11. HelloUDP

1. Реализуйте клиент и сервер, взаимодействующие по UDP.
2. Класс `HelloUDPClient` должен отправлять запросы на сервер, принимать результаты и выводить их на консоль.
   - Аргументы командной строки:
     1. имя или ip-адрес компьютера, на котором запущен сервер;
     2. номер порта, на который отсылать запросы;
     3. префикс запросов (строка);
     4. число параллельных потоков запросов;
     5. число запросов в каждом потоке.
   - Запросы должны одновременно отсылаться в указанном числе потоков. Каждый поток должен ожидать обработки своего запроса и выводить сам запрос и результат его обработки на консоль. Если запрос не был обработан, требуется послать его заново.
   - Запросы должны формироваться по схеме `<префикс запросов><номер запроса в потоке>_<номер потока>`.
3. Класс `HelloUDPServer` должен принимать задания, отсылаемые классом `HelloUDPClient` и отвечать на них.
   - Аргументы командной строки:
     1. номер порта, по которому будут приниматься запросы;
     2. число рабочих потоков, которые будут обрабатывать запросы.
   - Ответом на запрос должно быть `Hello, <текст запроса>`.
   - Несмотря на то, что текущий способ получения ответа по запросу очень прост, сервер должен быть рассчитан на ситуацию, когда этот процесс может требовать много ресурсов и времени.
   - Если сервер не успевает обрабатывать запросы, прием запросов может быть временно приостановлен.

Библиография

1. Java Networking
2. Custom Networking (tutorial)
3. Uniform Resource Locators (URL)
4. Uniform Resource Identifiers (URI)

Тема 13. Сериализация и RMI

Содержание

1. Сериализация
   1. Сериализация и десериализация
   2. Объектные потоки
   3. Автоматическая сериализация
   4. Ручная сериализация
   5. Настраиваемая сериализация
   6. Сериализация и версионирование
   7. Сериализация и дескрипторы
2. Концепции RMI
   1. Предназначение
   2. Схема работы и стек протоколов
   3. Удаленные интерфейсы
   4. Stub и Skeleton
   5. Распределенная сборка мусора
   6. RMI Registry
3. Применение RMI

Слайды

HTML

Примеры

zip

Домашнее задание 12. Физические лица

1. Добавьте к банковскому приложению возможность работы с физическими лицами.
   1. У физического лица (`Person`) можно запросить имя, фамилию и номер паспорта.
   2. Удалённые физические лица (`RemotePerson`) должны передаваться при помощи удалённых объектов.
   3. Локальные физические лица (`LocalPerson`) должны передаваться при помощи механизма сериализации, и при последующем использовании не требовать связи с сервером.
   4. Должна быть возможность поиска физического лица по номеру паспорта, с выбором типа возвращаемого лица.
   5. Должна быть возможность создания записи о физическом лице по его данным.
2. У физического лица может быть несколько счетов, к которым должен предоставляться доступ через `Person`.
   1. Счёту физического лица с идентификатором *subId* должен соответствовать банковский счёт с *id* вида *personId:subId*.
   2. Изменения, производимые со счётом в банке (создание и изменение баланса), должны быть видны всем соответствующим `RemotePerson`, и только тем `LocalPerson`, которые были созданы после этого изменения.
   3. Изменения в счетах, производимые через `RemotePerson`, должны сразу применяться глобально, а производимые через `LocalPerson` – только локально для этого конкретного `LocalPerson`.
3. Реализуйте приложение, демонстрирующее работу с физическим лицами.
   1. Аргументы командной строки: имя, фамилия, номер паспорта физического лица, номер счёта, изменение суммы счёта.
   2. Если информация об указанном физическом лице отсутствует, то оно должно быть добавлено. В противном случае – должны быть проверены его данные.
   3. Если у физического лица отсутствует счёт с указанным номером, то он создается с нулевым балансом.
   4. После обновления суммы счёта новый баланс должен выводиться на консоль.
4. **Сложный вариант**
   1. На каждом счету всегда должно быть неотрицательное количество денег.
5. Приложение должно находиться в пакете `info.kgeorgiy.ja.*.bank` и его подпакетах.

Домашнее задание 13. Физические лица (тесты)

1. Напишите тесты, проверяющие поведение банка и приложения из домашнего задания Физические лица.
   - Для реализации тестов рекомендуется использовать JUnit (Tutorial). Множество примеров использования можно найти в тестах.
   - Если вы знакомы с другим тестовым фреймворком (например, TestNG), то можете использовать его.
   - Добавьте jar-файлы используемых библиотек в каталог `lib` вашего репозитория.
   - *Нельзя* использовать самописные фреймворки и тесты, запускаемые через `main`.
2. **Сложный вариант**
   1. Тесты не должны рассчитывать на наличие запущенного RMI Registry.
   2. Создайте класс `BankTests`, запускающий тесты.
   3. Создайте скрипт, запускающий `BankTests` и возвращающий код (статус) `0` в случае успеха и `1` в случае неудачи.
   4. Создайте скрипт, запускающий тесты с использованием стандартного подхода для вашего тестового фреймворка. Код возврата должен быть как в предыдущем пункте.
3. Тесты должны находиться в пакете `info.kgeorgiy.ja.*.bank` и его подпакетах.
4. При сдаче вам нужно будет продемонстрировать работу тестов на вашем компьютере.

Это домашнее задание **связано** с домашним заданием Физические лица и будет приниматься только с ним.

Библиография

1. Java Object Serialization Specification
2. Java Object Serialization (Guide)
3. Serialization FAQ
4. Java RMI Specification
5. The Java Remote Method Invocation API
6. RMI Tutorial

Тема 14. Неблокирующий и асинхронный ввод-вывод

Содержание

1. Буферы
   1. Операции
   2. Чтение и запись
   3. Байтовые буферы
   4. Типизированные буферы
2. Каналы
   1. Иерархия каналов
   2. Байтовые каналы
   3. Файловые каналы
      1. Файлы отображаемые в память
      2. Блокировки
   4. Сетевые каналы
   5. Неблокирующий ввод-вывод
   6. Асинхронные каналы

Слайды

HTML

Примеры

zip

Домашнее задание 14. HelloNonblockingUDP

1. Реализуйте клиент и сервер, взаимодействующие по UDP, используя только неблокирующий ввод-вывод.
2. Класс `HelloUDPNonblockingClient` должен иметь функциональность аналогичную `HelloUDPClient`, но без создания новых потоков.
3. Класс `HelloUDPNonblockingServer` должен иметь функциональность аналогичную `HelloUDPServer`, но все операции с сокетом должны производиться в одном потоке.
4. В реализации не должно быть активных ожиданий, в том числе через `Selector`.
5. Обратите внимание на выделение общего кода старой и новой реализации.
6. Можно использовать неблокирующий или асинхронный ввод-вывод. Нельзя использовать виртуальные потоки.
7. *Бонусный вариант*. Клиент и сервер могут перед началом работы выделить *O(число рабочих потоков)* памяти. Выделять дополнительную память во время работы запрещено.

Библиография

1. Java NIO
2. Java Tutorial. Basic I/O
3. Java™ I/O, NIO, and NIO.2
4. Java NIO Selector
5. Java Asynchronous I/O
6. Virtual Threads

Часть 5. Разное

Тем: 2

Тема 15. Интернационализация и локализация

Содержание

1. Локали
2. Локализация данных
   1. Создание ResourceBundle
   2. Загрузка ResourceBundle
   3. Использование ResourceBundle
3. Форматирование
   1. Форматирование чисел
   2. Форматирование дат и времени
   3. Форматирование сообщений
4. Работа с текстом
   1. Сравнение строк
   2. Разбиение текста
   3. Запись текста

Слайды

HTML

Примеры

zip

Домашнее задание 15. Статистика текста

1. Создайте приложение `TextStatistics`, анализирующее тексты на различных языках.
   1. Аргументы командной строки:
      - локаль текста,
      - локаль вывода,
      - файл с текстом,
      - файл отчета.
   2. Поддерживаемые локали текста: все локали, имеющиеся в системе.
   3. Поддерживаемые локали вывода: русская и английская.
   4. Файлы имеют кодировку UTF-8.
   5. Подсчет статистики должен вестись по следующим категориям:
      - предложения,
      - слова,
      - числа,
      - деньги,
      - даты.
   6. Для каждой категории должна собираться следующая статистика:
      - число вхождений,
      - число различных значений,
      - минимальное значение,
      - максимальное значение,
      - минимальная длина,
      - максимальная длина,
      - среднее значение/длина.
   7. Пример отчета:
      ```
      Анализируемый файл "quine.ru-RU.in".
      Сводная статистика
          Число предложений: 30.
          Число слов: 120.
          Число чисел: 36.
          Число сумм: 3.
          Число дат: 3.
      Статистика по предложениям
          Число предложений: 30 (30 различных).
          Минимальное предложение: "Анализируемый файл "quine.ru-RU.in".".
          Максимальное предложение: "Число чисел: 36.".
          Минимальная длина предложения: 13 ("Число дат: 3.").
          Максимальная длина предложения: 127 ("GK: если сюда вставить реальное предложение и не вставить 11, то процесс не сойдётся!!").
          Средняя длина предложения: 38,367.
      Статистика по словам
          Число слов: 120 (48 различных).
          Минимальное слово: "GK".
          Максимальное слово: "языках".
          Минимальная длина слова: 1 ("и").
          Максимальная длина слова: 18 ("TextStatisticsTest").
          Средняя длина слова: 6,667.
      Статистика по числам
          Число чисел: 36 (23 различных).
          Минимальное число: -12345,67.
          Максимальное число: 23456,78.
          Среднее число: 528,914.
      Статистика по суммам денег
          Число сумм: 3 (3 различных).
          Минимальная сумма: 100,00 ₽.
          Максимальная сумма: 234,56 ₽.
          Средняя сумма: 167,28 ₽.
      Статистика по датам
          Число дат: 3 (3 различных).
          Минимальная дата: 22 мая 2026 г..
          Максимальная дата: 29 мая 2026 г..
          Средняя дата: 25 мая 2026 г..
      ```
2. Вы можете рассчитывать на то, что весь текст помещается в память.
3. При выполнении задания следует обратить внимание на:
   1. Декомпозицию сообщений для локализации.
   2. Согласование сообщений по роду и числу.
4. Приложение должно находиться в пакете `info.kgeorgiy.ja.*.i18n` и его подпакетах.

Тема 16. Сборка мусора

Содержание

1. Сборка мусора
   1. Сжимающие и копирующие сборщики
   2. Гипотеза о поколениях и ее применение
2. Ссылки
   1. Типы ссылок
   2. Очереди ссылок

Слайды

HTML

Примеры

zip

Библиография

1. Garbage Collection
2. Understanding Weak References
3. Package java.lang.ref
4. HotSpot VM GC Tuning guide
5. Shenandoah GC
6. Deep Dive into ZGC

Часть 6. Интерфейс пользователя

Тем: 4

Тема 17. Стандартные компоненты

Примеры

zip

Слайды

HTML

Тема 18. Обработка событий

Слайды

HTML

Примеры

zip

Тема 19. Модели

Слайды

HTML

Примеры

zip

Тема 20. JavaBeans

Слайды

HTML

Часть 7. XML

Тем: 5

Тема 21. Введение в XML

Слайды

PDF

Тема 22. Описание и проверка структуры XML

Слайды

PDF

Тема 23. Apache Ant

Слайды

PDF

Тема 24. XPath

Слайды

PDF

Тема 25. XSLT

Слайды

PDF

Часть 1. Введение
Тема 1. Ввод-вывод
Содержание
Слайды
Примеры
Домашнее задание 1. Обход файлов
Библиография
Тема 2. Collections Framework
Содержание
Слайды
Примеры
Домашнее задание 2. Множество на массиве
Библиография
Тема 3. Лямбда-выражения и потоки
Содержание
Слайды
Примеры
Домашнее задание 3. Студенты
Домашнее задание 4. Реализация потоков
Библиография
Тема 4. Reflection
Содержание
Слайды
Примеры
Домашнее задание 5. Implementor
Библиография
Часть 2. Инструменты
Тема 5. Jar
Содержание
Слайды
Домашнее задание 6. Jar Implementor
Библиография
Тема 6. Javadoc
Содержание
Слайды
Примеры
Домашнее задание 7. Javadoc
Библиография
Тема 7. Модули
Содержание
Слайды
Примеры
Библиография
Часть 3. Многопоточное программирование
Тема 8. Введение в многопоточное программирование
Содержание
Слайды
Примеры
Домашнее задание 8. Итеративный параллелизм
Библиография
Тема 9. Взаимодействие потоков
Содержание
Слайды
Примеры
Домашнее задание 9. Параллельный запуск
Библиография
Тема 10. Задачи и средства многопоточного программирования
Содержание
Слайды
Библиография
Тема 11. Concurrency Utilities
Содержание
Слайды
Домашнее задание 10. Web Crawler
Библиография
Часть 4. Сетевые приложения
Тема 12. Работа с сетью
Содержание
Слайды
Примеры
Домашнее задание 11. HelloUDP
Библиография
Тема 13. Сериализация и RMI
Содержание
Слайды
Примеры
Домашнее задание 12. Физические лица
Домашнее задание 13. Физические лица (тесты)
Библиография
Тема 14. Неблокирующий и асинхронный ввод-вывод
Содержание
Слайды
Примеры
Домашнее задание 14. HelloNonblockingUDP
Библиография
Часть 5. Разное
Тема 15. Интернационализация и локализация
Содержание
Слайды
Примеры
Домашнее задание 15. Статистика текста
Тема 16. Сборка мусора
Содержание
Слайды
Примеры
Библиография
Часть 6. Интерфейс пользователя
Тема 17. Стандартные компоненты
Примеры
Слайды
Тема 18. Обработка событий
Слайды
Примеры
Тема 19. Модели
Слайды
Примеры
Тема 20. JavaBeans
Слайды
Часть 7. XML
Тема 21. Введение в XML
Слайды
Тема 22. Описание и проверка структуры XML
Слайды
Тема 23. Apache Ant
Слайды
Тема 24. XPath
Слайды
Тема 25. XSLT
Слайды